Latency Minimization for Uplink RSMA with Dynamic Sub-Block Allocation

June 10, 2026

Abstract

This report presents a cleaned and self-contained formulation of the final algorithmic outcome of the project. The objective is to transmit the payload of each uplink user with minimum latency when the number of required sub-blocks is not known in advance. For each user, the method jointly determines the number of sub-blocks, the bits carried by each sub-block, the blocklength used in each sub-block, and the corresponding precoder, subject to finite-blocklength reliability and power constraints. The final method consists of four layers: an effective finite-blocklength rate model, a global latency minimization problem with unknown sub-block count, a greedy outer loop that constructs sub-blocks dynamically, and an inner alternating-optimization routine with a primal-dual precoder update.

1 Introduction

The final version of the work can be summarized as follows. Each user $k \in \mathcal{K}$ has a total payload of B_k information bits that must be delivered to the base station. A single transmission block is no longer assumed to be known beforehand. Instead, the payload may be split across multiple sequential sub-blocks, where the number of such sub-blocks, denoted by L_k, is itself part of the solution. The design goal is to minimize uplink latency while ensuring that every transmitted sub-block satisfies a finite-blocklength decoding constraint and a transmit-power constraint.

Earlier exploratory material considered several intermediate formulations. Those derivations are intentionally omitted here. The cleaned report focuses only on the final formulation that survived the development process:

1. represent each user sub-block by an effective finite-blocklength rate,
2. optimize latency with L_k unknown,
3. construct sub-blocks greedily from the remaining bits, and
4. solve the per-sub-block design by alternating between blocklength search and precoder optimization.

2 Notation and Effective Physical-Layer Model

2.1 Indices, Sets, and Variables

The user set is

$$\mathcal{K} = \{1, 2, \ldots, K\},$$

where K is the number of uplink users.

The most important symbols are summarized in Table 1. Every symbol used later in the report is defined either in the table or at first use.

2.2 Effective Per-Sub-Block Model

The cleaned formulation is intentionally written at the level of an *effective* user-specific channel. This keeps the final algorithm independent of the exploratory derivations that appeared earlier in the project. Any uplink access architecture, including RSMA, may be used in the background as long as it yields an effective channel matrix $\mathbf{H}_{k,\ell}$ and an achievable finite-blocklength rate model for each user and each sub-block.

For the AWGN MIMO subproblem used in the final algorithm, define

$$\mathbf{A}_{k,\ell}(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell}) = \sigma^{-2}\mathbf{H}_{k,\ell}\mathbf{F}_{k,\ell}\mathbf{F}_{k,\ell}^H\mathbf{H}_{k,\ell}^H, \tag{1}$$

$$C_{k,\ell}(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell}) = \log_2 \det\big(\mathbf{I} + \mathbf{A}_{k,\ell}(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell})\big), \tag{2}$$

$$V_{k,\ell}(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell}) = (\log_2 e)^2 \operatorname{tr}\Big[\mathbf{A}_{k,\ell}(\mathbf{A}_{k,\ell}+2\mathbf{I})(\mathbf{A}_{k,\ell}+\mathbf{I})^{-2}\Big]. \tag{3}$$

In (3), the dependence of the last line on $(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell})$ is omitted inside the trace only to keep the notation compact.

Using the normal approximation for finite blocklength [1], the achievable rate of sub-block (k,ℓ) is

$$R^*_{k,\ell}(n_{k,\ell},\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell},\varepsilon_k) = C_{k,\ell}(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell}) - \sqrt{\frac{V_{k,\ell}(\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell})}{n_{k,\ell}}}\,Q^{-1}(\varepsilon_k). \tag{4}$$

The finite-blocklength decoding constraint for a sub-block that carries $B_{k,\ell}$ bits is therefore

$$\frac{B_{k,\ell}}{n_{k,\ell}} \le R^*_{k,\ell}(n_{k,\ell},\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell},\varepsilon_k). \tag{5}$$

3 Latency Minimization with Unknown Number of Sub-Blocks

3.1 Design Objective

The final optimization problem chooses, for every user k, the number of sub-blocks L_k, the bit allocation $\{B_{k,\ell}\}_{\ell=1}^{L_k}$, the blocklengths $\{n_{k,\ell}\}_{\ell=1}^{L_k}$, and the precoders $\{\mathbf{F}_{k,\ell}\}_{\ell=1}^{L_k}$ so as to minimize the total uplink latency

$$\sum_{k\in\mathcal{K}} \tau_k = \sum_{k\in\mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell=1}^{L_k} n_{k,\ell}. \tag{6}$$

3.2 Optimization Problem

The cleaned final problem is

$$\min_{\{L_k,B_{k,\ell},n_{k,\ell},\mathbf{F}_{k,\ell}\}} \quad \sum_{k\in\mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell=1}^{L_k} n_{k,\ell} \tag{7a}$$

$$\text{s.t.} \quad \frac{B_{k,\ell}}{n_{k,\ell}} \le R^*_{k,\ell}(n_{k,\ell},\mathbf{H}_{k,\ell},\mathbf{F}_{k,\ell},\varepsilon_k), \qquad \forall k\in\mathcal{K},\ \forall \ell\in\{1,\dots,L_k\}, \tag{7b}$$

$$\sum_{\ell=1}^{L_k} B_{k,\ell} = B_k, \qquad \forall k\in\mathcal{K}, \tag{7c}$$

$$1 \le n_{k,\ell} \le T_k, \qquad \forall k\in\mathcal{K},\ \forall \ell\in\{1,\dots,L_k\}, \tag{7d}$$

$$B_{k,\ell}\in\mathbb{Z}_{\ge 0}, \qquad L_k\in\mathbb{Z}_{\ge 1}, \tag{7e}$$

$$\|\mathbf{F}_{k,\ell}\|_F^2 \le P_{k,\ell}, \qquad \forall k\in\mathcal{K},\ \forall \ell\in\{1,\dots,L_k\}. \tag{7f}$$

Problem (7a)–(7f) is difficult because:

1. the number of sub-blocks L_k is not known in advance,
2. the rate constraint couples $B_{k,\ell}$, $n_{k,\ell}$, and $\mathbf{F}_{k,\ell}$ nonlinearly, and
3. the finite-blocklength rate in (4) is nonconvex in the joint design variables.

The final algorithm handles these difficulties by constructing the sub-blocks sequentially.

4 Greedy Dynamic Construction of Sub-Blocks

4.1 Two Core Building Blocks

For each user k and a candidate payload B to be sent in a single coherence sub-block, define the per-sub-block design operator

$$\mathrm{AO}_k(B, T_k, \varepsilon_k) = (n_k^\star(B), \mathbf{F}_k^\star(B)), \tag{8}$$

where $(n_k^\star(B), \mathbf{F}_k^\star(B))$ is obtained by solving the minimum-blocklength design problem

$$\begin{aligned}
\min_{n,\mathbf{F}} \quad & n && \text{(9a)}\\
\text{s.t.} \quad & \frac{B}{n} \le R_{k,\ell}^{*}(n, \mathbf{H}_{k,\ell}, \mathbf{F}, \varepsilon_k), && \text{(9b)}\\
& 1 \le n \le T_k, && \text{(9c)}\\
& \|\mathbf{F}\|_F^2 \le P_{k,\ell}. && \text{(9d)}
\end{aligned}$$

The second building block is the maximum number of bits that can be carried by a fixed pair $(n, \mathbf{F})$:

$$B_{k,\ell}^\star(n, \mathbf{H}_{k,\ell}, \mathbf{F}, \varepsilon_k) = \left\lfloor n\, R_{k,\ell}^{*}(n, \mathbf{H}_{k,\ell}, \mathbf{F}, \varepsilon_k) \right\rfloor. \tag{10}$$

Equation (10) is the largest integer payload that satisfies the finite-blocklength constraint in (5) for the given blocklength and precoder.

4.2 Greedy Outer Algorithm

The sequential construction starts from the total remaining payload of a user and creates one sub-block at a time.

Algorithm 1 Greedy Dynamic Sub-Block Construction for User k

Require: Total payload B_k, coherence blocklength T_k, error target ε_k

Ensure: $\{(n_{k,\ell}^\star, \mathbf{F}_{k,\ell}^\star, B_{k,\ell}^{\mathrm{tx}})\}_{\ell=1}^{L_k}$

```
1: ℓ ← 1
2: B_{k,rem}^{(1)} ← B_k
3: while B_{k,rem}^{(ℓ)} > 0 do
4:     (n*_{k,ℓ}, F*_{k,ℓ}) ← AO_k(B_{k,rem}^{(ℓ)}, T_k, ε_k)
5:     B_{k,ℓ}^{tx} ← min{ B_{k,rem}^{(ℓ)}, B*_{k,ℓ}(n*_{k,ℓ}, H_{k,ℓ}, F*_{k,ℓ}, ε_k) }
6:     B_{k,rem}^{(ℓ+1)} ← B_{k,rem}^{(ℓ)} − B_{k,ℓ}^{tx}
7:     ℓ ← ℓ + 1
8: L_k ← ℓ − 1
```

The logic of Algorithm 1 is simple:

1. First, try to send all remaining bits in one sub-block using the smallest feasible blocklength.

2. If that is impossible, the inner routine returns the best feasible design within the coherence limit T_k.

3. The quantity $B^{\mathrm{tx}}_{k,\ell}$ then becomes the maximum payload that can actually be carried by the selected sub-block.

4. The leftover bits are passed to the next sub-block, and the process repeats.

Hence L_k is no longer a simulation input. It is the output of the loop termination condition

$$B^{(L_k+1)}_{k,\mathrm{rem}} = 0.$$

The algorithm is well-defined whenever each serviceable user can transmit at least one bit in a coherence sub-block under its power budget; otherwise the user-side problem is infeasible.

5 Alternating Optimization for the Single-Sub-Block Problem

5.1 Why Alternating Optimization is Used

The subproblem (9a)–(9d) is nonconvex in $(n, \mathbf{F})$ jointly. The final method therefore alternates between:

1. updating the blocklength n for a fixed precoder, and

2. updating the precoder $\mathbf{F}$ for a fixed blocklength.

5.2 Blocklength Update for Fixed Precoder

Suppose the current precoder at inner iteration i is $\mathbf{F}^{(i)}_{k,\ell}$. The next blocklength is chosen as the smallest feasible integer

$$n^{(i+1)}_{k,\ell} = \min\left\{ n \in \{1, \ldots, T_k\} : \frac{B}{n} \le R^*_{k,\ell}(n, \mathbf{H}_{k,\ell}, \mathbf{F}^{(i)}_{k,\ell}, \varepsilon_k) \right\}. \tag{11}$$

If the feasible set in (11) is empty, then the target payload B cannot be placed into a single sub-block under the coherence limit. In that case the algorithm uses

$$n^{(i+1)}_{k,\ell} = T_k. \tag{12}$$

5.3 Precoder Update for Fixed Blocklength

For a fixed blocklength $n^{(i+1)}_{k,\ell}$, the precoder step maximizes the finite-blocklength rate. Equivalently, it makes the payload constraint as easy to satisfy as possible:

$$\mathbf{F}^{(i+1)}_{k,\ell} = \arg\max_{\mathbf{F}} \quad R^*_{k,\ell}(n^{(i+1)}_{k,\ell}, \mathbf{H}_{k,\ell}, \mathbf{F}, \varepsilon_k) \tag{13a}$$

$$\text{s.t.} \quad \|\mathbf{F}\|_F^2 \le P_{k,\ell}. \tag{13b}$$

Because (13a)–(13b) is still nonconvex, the final implementation parameterizes the precoder and solves the update numerically.

Algorithm 2 Alternating Optimization for $\mathrm{AO}_k(B, T_k, \varepsilon_k)$

Require: Target payload B, coherence blocklength T_k, error target ε_k
Ensure: $(n^\star_{k,\ell}, \mathbf{F}^\star_{k,\ell})$

1: Initialize $\mathbf{F}^{(0)}_{k,\ell}$ and $n^{(0)}_{k,\ell}$
2: **for** $i = 0, 1, \ldots, I_{\max} - 1$ **do**
3: Update $n^{(i+1)}_{k,\ell}$ using (11), with fallback (12)
4: Update $\mathbf{F}^{(i+1)}_{k,\ell}$ by solving (13a)–(13b)
5: $(n^\star_{k,\ell}, \mathbf{F}^\star_{k,\ell}) \leftarrow (n^{(I_{\max})}_{k,\ell}, \mathbf{F}^{(I_{\max})}_{k,\ell})$

5.4 Inner Alternating-Optimization Routine

6 Neural Primal-Dual Update for the Precoder Step

6.1 Parameterized Precoder

The final formulation represents the precoder by a trainable mapping

$$\mathbf{F}_{k,\ell} = f_{\theta_{k,\ell}}(\mathbf{H}_{k,\ell}), \tag{14}$$

where $f_{\theta_{k,\ell}}(\cdot)$ may be any neural architecture that outputs a complex precoder matrix and $\theta_{k,\ell}$ denotes its trainable parameters.

6.2 Constraint Residuals

For a fixed blocklength $n_{k,\ell}$, define the finite-blocklength rate residual

$$c_{1,k,\ell}(\theta_{k,\ell}) = \frac{B_{k,\ell}}{n_{k,\ell}} - R^*_{k,\ell}(n_{k,\ell}, \mathbf{H}_{k,\ell}, f_{\theta_{k,\ell}}(\mathbf{H}_{k,\ell}), \varepsilon_k), \tag{15}$$

and the power residual

$$c_{2,k,\ell}(\theta_{k,\ell}) = \left\| f_{\theta_{k,\ell}}(\mathbf{H}_{k,\ell}) \right\|_F^2 - P_{k,\ell}. \tag{16}$$

The finite-blocklength constraint is satisfied when $c_{1,k,\ell}(\theta_{k,\ell}) \leq 0$, and the power constraint is satisfied when $c_{2,k,\ell}(\theta_{k,\ell}) \leq 0$.

6.3 Lagrangian

Introduce nonnegative dual variables $\lambda_{1,k,\ell} \geq 0$ and $\lambda_{2,k,\ell} \geq 0$. The clean Lagrangian for the fixed-blocklength precoder subproblem is

$$\mathcal{L}_{k,\ell}(\theta_{k,\ell}, \lambda_{1,k,\ell}, \lambda_{2,k,\ell}) = -R^*_{k,\ell}(n_{k,\ell}, \mathbf{H}_{k,\ell}, f_{\theta_{k,\ell}}(\mathbf{H}_{k,\ell}), \varepsilon_k) + \lambda_{1,k,\ell} c_{1,k,\ell}(\theta_{k,\ell}) + \lambda_{2,k,\ell} c_{2,k,\ell}(\theta_{k,\ell}). \tag{17}$$

Minimizing (17) in $\theta_{k,\ell}$ while ascending in the dual variables encourages the precoder to increase the achievable rate while satisfying both constraints.

6.4 Primal-Dual Iterations

Let t denote the primal-dual iteration index. The parameter update is

$$\theta^{(t+1)}_{k,\ell} = \theta^{(t)}_{k,\ell} - \eta^{(t)}_\theta \nabla_{\theta_{k,\ell}} \mathcal{L}_{k,\ell}\big(\theta^{(t)}_{k,\ell}, \lambda^{(t)}_{1,k,\ell}, \lambda^{(t)}_{2,k,\ell}\big), \tag{18}$$

where $\eta^{(t)}_\theta > 0$ is the primal step size.

The dual variables are updated by projected ascent:

$$\lambda_{1,k,\ell}^{(t+1)} = \left[\lambda_{1,k,\ell}^{(t)} + \eta_{\lambda_1}^{(t)} c_{1,k,\ell}\left(\theta_{k,\ell}^{(t+1)}\right)\right]_+, \tag{19}$$

$$\lambda_{2,k,\ell}^{(t+1)} = \left[\lambda_{2,k,\ell}^{(t)} + \eta_{\lambda_2}^{(t)} c_{2,k,\ell}\left(\theta_{k,\ell}^{(t+1)}\right)\right]_+, \tag{20}$$

where

$$[x]_+ = \max\{x, 0\}. \tag{21}$$

The step sizes $\eta_{\lambda_1}^{(t)} > 0$ and $\eta_{\lambda_2}^{(t)} > 0$ are dual learning rates.

7 Interpretation of the Final Algorithm

The cleaned final procedure can be read in a straightforward way.

For each user, the algorithm first asks whether all remaining bits can fit into one coherence-limited sub-block. If the answer is yes, it chooses the smallest feasible blocklength and the matching precoder. If the answer is no, it uses the full coherence block, optimizes the precoder for that block, transmits as many bits as that block can support, and forwards the rest to the next sub-block. Repeating this logic generates the required number of sub-blocks automatically.

This interpretation explains why the number of sub-blocks L_k should not be fixed beforehand. It is a consequence of the payload size, the coherence constraint T_k, the power budget $P_{k,\ell}$, the effective channel $\mathbf{H}_{k,\ell}$, and the reliability target ε_k. In the final formulation, the unknown quantity L_k is therefore an output of the transmission design rather than a prescribed simulation parameter.

Table 1: Main notation

Symbol	Definition
$k \in \mathcal{K}$	User index.
$\ell \in \{1, \dots, L_k\}$	Sub-block index for user k.
K	Number of users.
L_k	Number of sub-blocks used by user k; this is unknown a priori and is determined by the algorithm.
B_k	Total payload of user k in bits.
$B_{k,\ell}$	Bits assigned to sub-block ℓ of user k.
$B_{k,\ell}^{\mathrm{tx}}$	Bits actually transmitted in sub-block ℓ by the greedy algorithm.
$B_{k,\mathrm{rem}}^{(\ell)}$	Remaining unsent bits of user k when iteration ℓ begins.
$n_{k,\ell}$	Blocklength of user k in sub-block ℓ, measured in channel uses or symbols.
T_k	Maximum available blocklength in one coherence sub-block for user k.
$f_{s,k}$	Symbol rate of user k in symbols per second.
τ_k	Total latency of user k, defined by $\tau_k = \sum_{\ell=1}^{L_k} n_{k,\ell}/f_{s,k}$.
ε_k	Target decoding error probability of user k.
$P_{k,\ell}$	Power budget for user k in sub-block ℓ.
N_r	Number of receive antennas at the base station.
N_t	Number of transmit antennas per user when a common transmit dimension is assumed.
d_k	Number of transmitted spatial streams of user k.
$\mathbf{H}_{k,\ell} \in \mathbb{C}^{N_r \times N_t}$	Effective channel matrix of user k in sub-block ℓ.
$\mathbf{F}_{k,\ell} \in \mathbb{C}^{N_t \times d_k}$	Precoder of user k in sub-block ℓ.
σ^2	Noise variance used in the effective AWGN model.
$C_{k,\ell}(\cdot)$	Effective Shannon capacity in bits per channel use.
$V_{k,\ell}(\cdot)$	Channel dispersion in bits^2 per channel use.
$R_{k,\ell}^{*}(\cdot)$	Normal-approximation finite-blocklength rate in bits per channel use.
$Q^{-1}(\cdot)$	Inverse Gaussian Q-function.

8 Results

This section collects the final experiment figures corresponding to the dynamic sub-block allocation algorithm described above. The first experiment is a representative four-user case that makes the behavior of the greedy splitting procedure easy to inspect. The second experiment scales the same procedure to one hundred users and shows that the latency reduction and synchronization improvement remain visible at larger system size.

8.1 Representative Four-User Experiment

The four-user experiment is used to visualize how the final algorithm reacts to heterogeneous payload, power, and coherence constraints. Figure 1 summarizes the user configuration. Users with lighter payloads or more favorable resource budgets can be served in fewer sub-blocks, while heavier users may need multiple sequential sub-blocks before the remaining payload becomes small enough to fit into a shorter final block.

Figure 2 shows the selected sub-blocklengths and corresponding finite-blocklength operating points for the four users. Users 0 and 1 terminate after one sub-block, whereas users 2 and 3 are split across multiple sub-blocks. This is consistent with the role of the outer greedy loop: once a full payload cannot be placed efficiently in a single coherence-limited block, the algorithm transmits the largest feasible portion and forwards the remainder to the next block.

The system-level effect is shown in Figure 3. The left panel compares initial and final latency user by user, while the right panel shows the latency histogram and empirical cumulative distribution. The final latency values move leftward relative to the initial ones, indicating that the optimized schedule consistently shortens transmission time across the user set.

Figure 4 focuses on synchronization. The asynchronality distribution becomes more concentrated near smaller values after optimization, and the pairwise heatmaps show that the off-diagonal latency gaps are compressed. This illustrates the intended effect of the method: users still finish at different times, but the spread in finishing times is reduced.

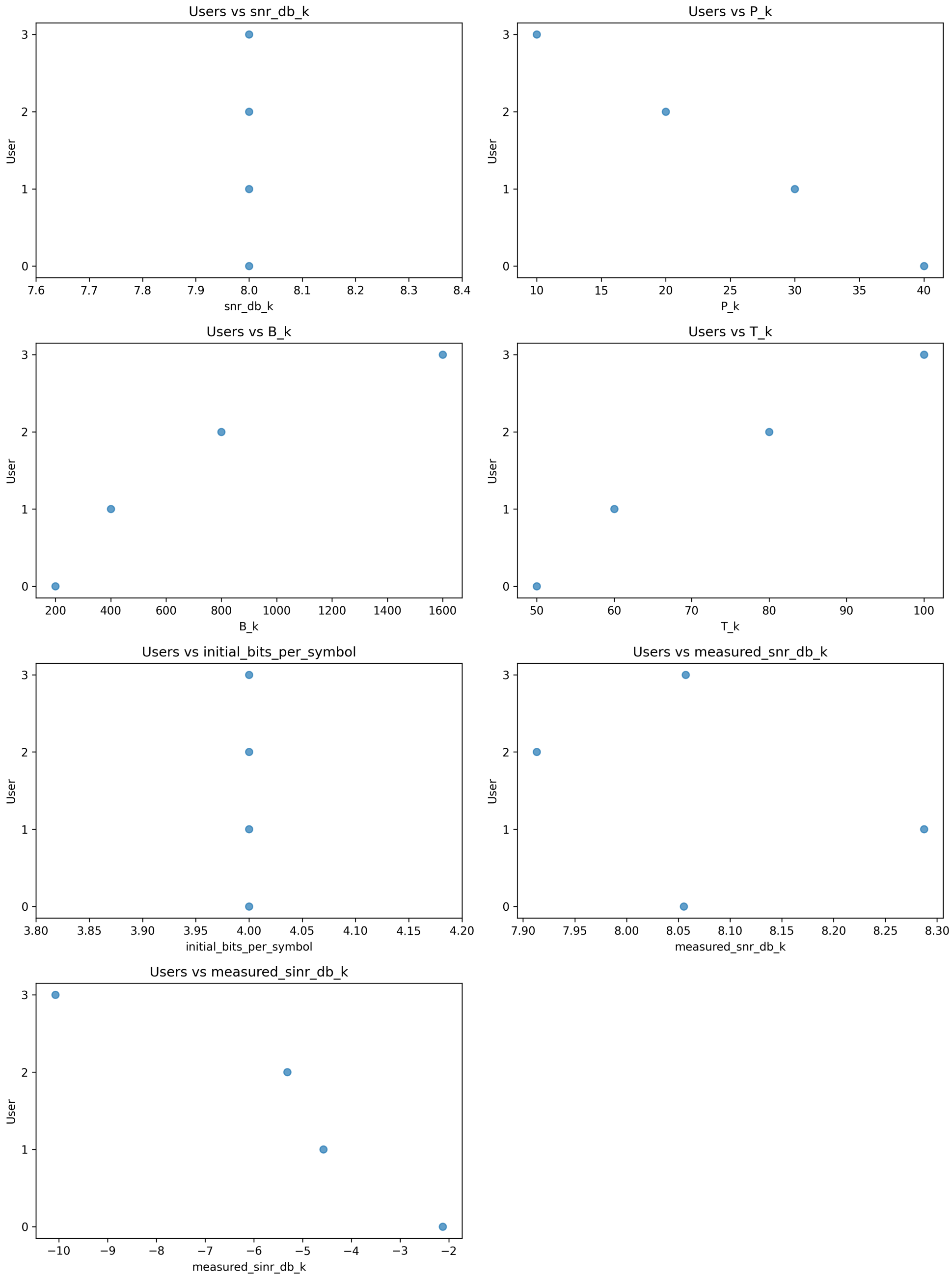
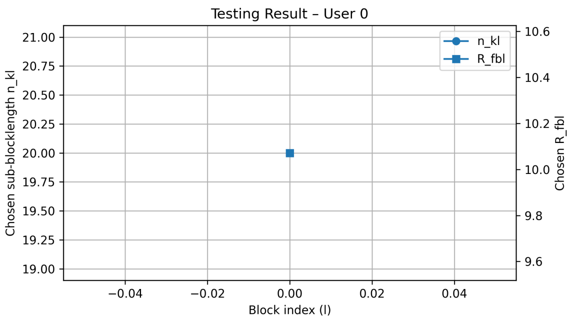
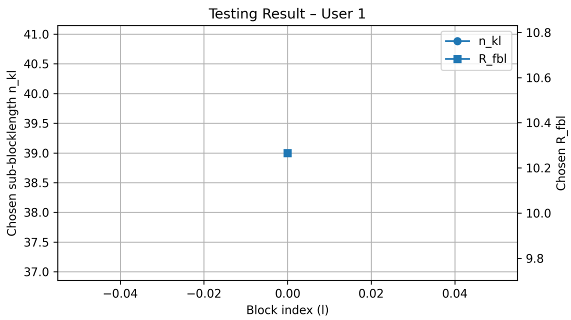


Figure 1: Configuration summary for the representative four-user experiment. The figure shows the per-user SNR, power budget, payload size, coherence blocklength, initial modulation choice, and measured link quality used by the final algorithm.

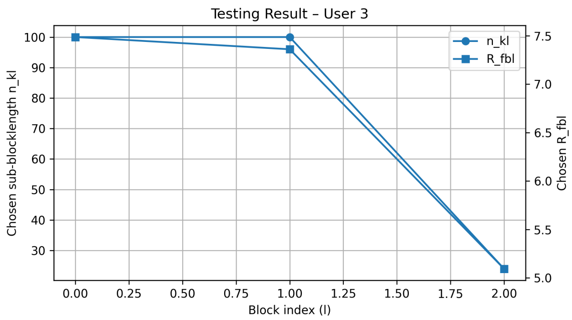


(a) User 0



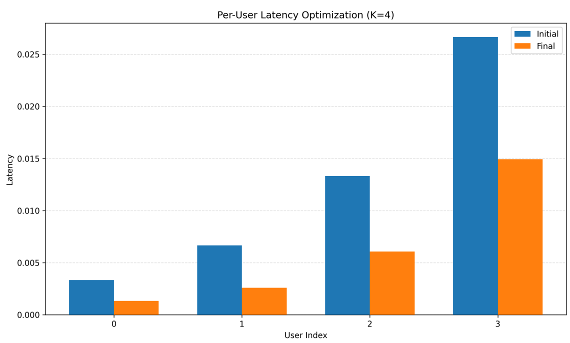
(b) User 1

(c) User 2

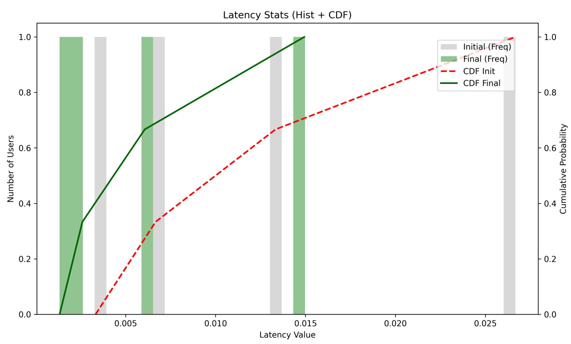


(d) User 3

Figure 2: Per-user outcomes in the representative four-user experiment. Each panel reports the chosen sub-blocklength trajectory and the corresponding finite-blocklength rate across the sub-blocks created by the greedy outer loop.

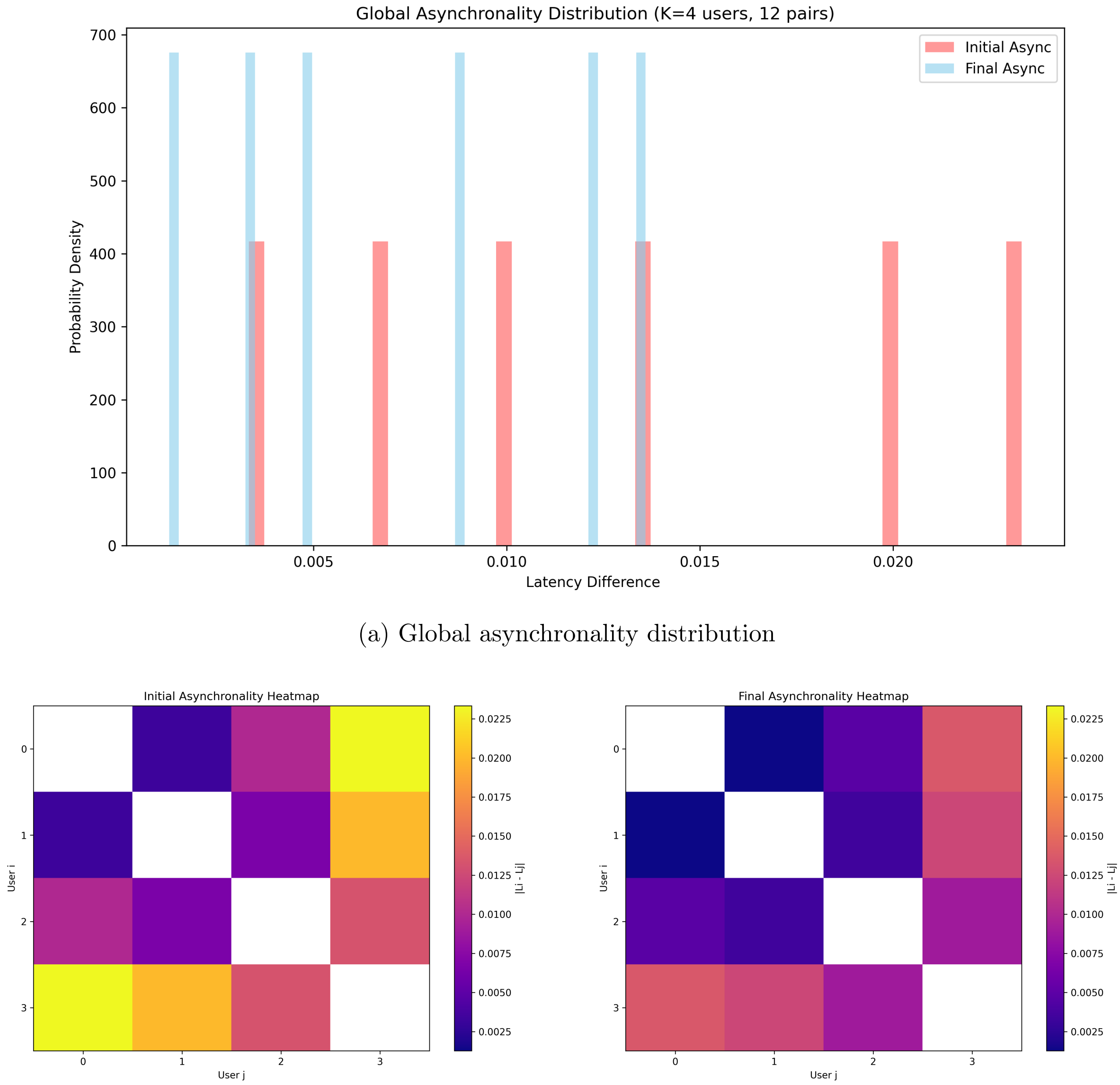


(a) Per-user latency before and after optimization



(b) Latency histogram and empirical CDF

Figure 3: Latency reduction for the representative four-user experiment.



(a) Global asynchronality distribution

(b) Initial pairwise asynchronality

(c) Final pairwise asynchronality

Figure 4: Asynchronality behavior in the representative four-user experiment.

8.2 Large-Scale Hundred-User Experiment

The hundred-user experiment evaluates the same algorithm under significantly more heterogeneous system conditions. Figure 5 summarizes the resulting spread of SNR, power budget, payload size, coherence blocklength, and initial modulation level across users.

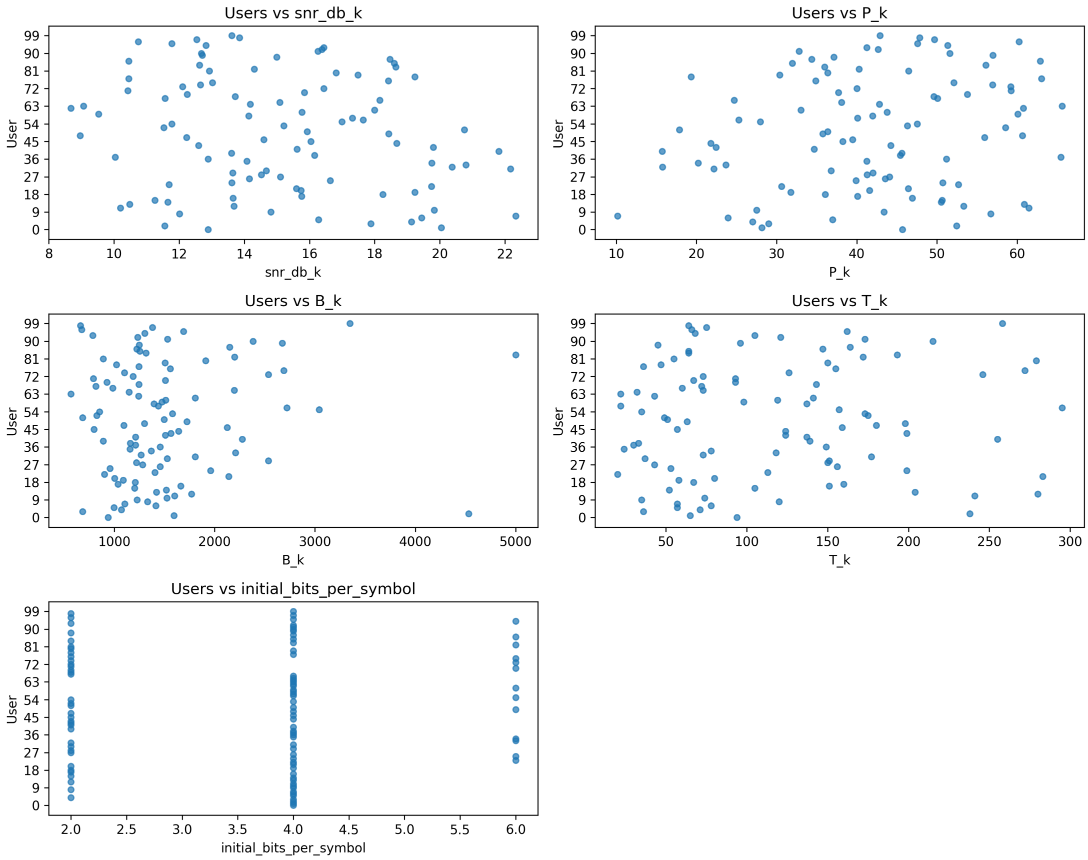
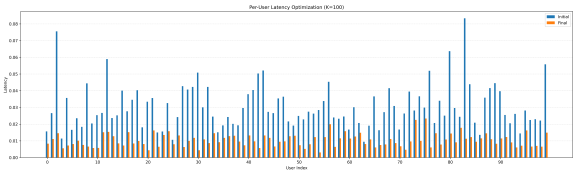


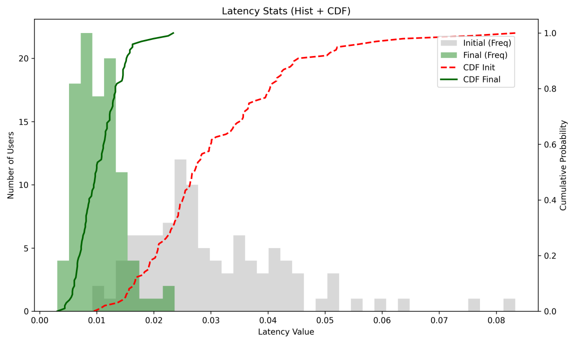
Figure 5: Configuration summary for the hundred-user experiment. The figure highlights the heterogeneity of user conditions that the final algorithm must handle.

Figure 6 shows the latency outcome at system scale. The per-user comparison confirms that the optimized schedule reduces latency for a broad range of users, and the histogram/CDF view shows a clear concentration of the final latency distribution at smaller values than the initial distribution.

Figure 7 shows the corresponding synchronization effect. The post-optimization distribution of pairwise latency differences is more concentrated near zero, and the final heatmap has visibly fewer bright structures than the initial one. Hence the same dynamic block construction that reduces total latency also improves global synchronization across a much larger user population.

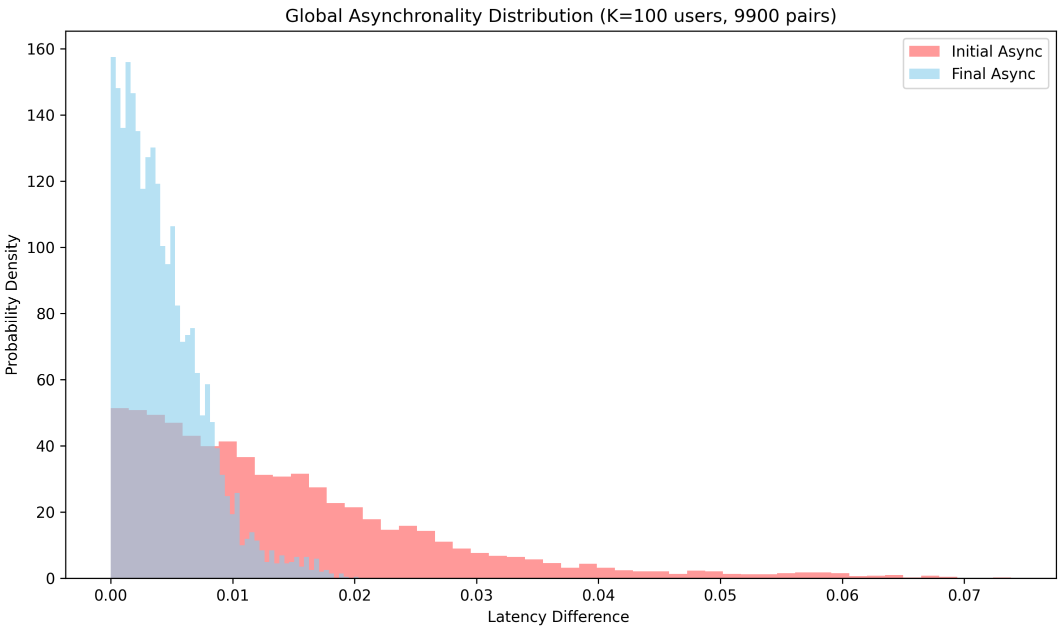


(a) Per-user latency before and after optimization

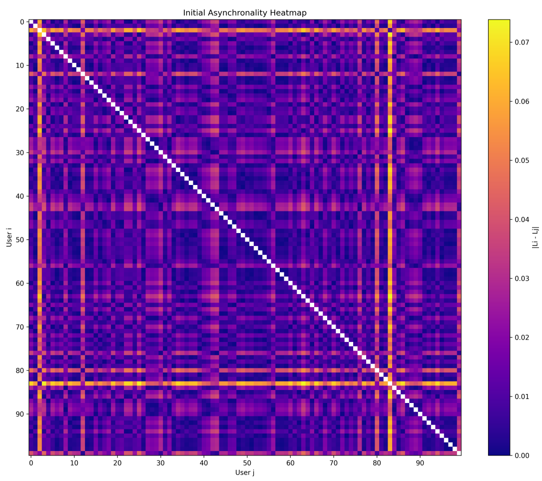


(b) Latency histogram and empirical CDF

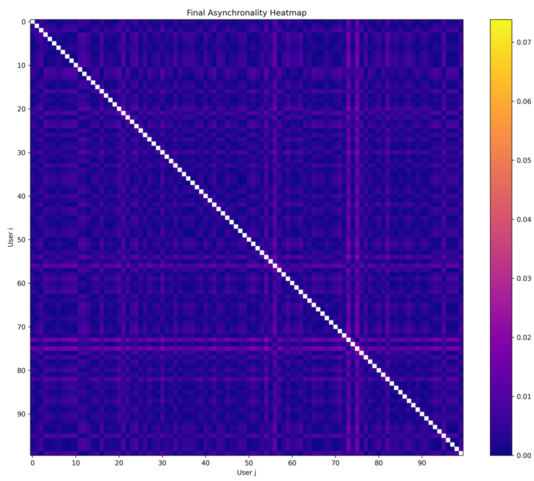
Figure 6: Latency reduction for the hundred-user experiment.



(a) Global asynchronality distribution



(b) Initial pairwise asynchronality



(c) Final pairwise asynchronality

Figure 7: Asynchronality behavior in the hundred-user experiment.

9 Conclusion

The final report reduces the project to one coherent optimization pipeline. The central problem is latency minimization with unknown sub-block count under finite-blocklength and power constraints. The solution is a greedy dynamic sub-block construction, supported by an alternating-optimization subroutine and a primal-dual neural precoder update. This cleaned formulation preserves the final algorithmic content of the project while removing the exploratory branches that were no longer needed.

References

[1] Y. Polyanskiy, H. V. Poor, and S. Verdu, "Channel coding rate in the finite blocklength regime," *IEEE Transactions on Information Theory*, vol. 56, no. 5, pp. 2307–2359, May 2010.

[2] Y. Zhang, W. Cheng, J. Wang, and W. Zhang, "Performance analysis and blocklength minimization of uplink RSMA for short packet transmissions in URLLC," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2023, pp. 6765–6770.

[3] B. Rimoldi and R. Urbanke, "A rate-splitting approach to the Gaussian multiple-access channel," *IEEE Transactions on Information Theory*, vol. 42, no. 2, pp. 364–375, Mar. 1996.

[4] Z. Yang, M. Chen, W. Saad, W. Xu, and M. Shikh-Bahaei, "Sum-rate maximization of uplink rate splitting multiple access communication," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.